

CELEBAL EXCELLENCE INTERNSHIP
PROGRAM

DATA ENGINEERING

WEEK - 3 ASSIGNMENT REPORT

SUBMITTED BY:-

ANUSHKA SHARMA

(JKLU)

- **STEP 1: SETUP DATA**

SQL QUERY:

```
CREATE DATABASE superstore;  
USE superstore;
```

```
CREATE TABLE customers AS  
SELECT DISTINCT  
Customer_ID,  
Customer_Name,  
Segment,  
Country,  
City,  
State,  
Postal_Code,  
Region  
FROM superstore_raw;
```

```
CREATE TABLE products AS  
SELECT DISTINCT  
Product_ID,  
Product_Name,  
Category,  
Sub_Category  
FROM superstore_raw;
```

```
CREATE TABLE orders AS
SELECT DISTINCT
Row_ID,
Order_ID,
Order_Date,
Ship_Date,
Ship_Mode,
Customer_ID,
Product_ID,
Sales,
Quantity,
Discount,
Profit
FROM superstore_raw;

ALTER TABLE orders
ADD PRIMARY KEY (Row_ID);
```

Explanation: The Superstore dataset was imported into superstore_raw. Separate **customers, products, and orders** tables were then created using **SELECT DISTINCT**.

Insight: Organizing the data into separate tables improves readability and makes sales analysis easier to perform.

● STEP 2: PERFORM REQUIRED QUERIES

SQL QUERY:

```
-- STEP 2
```

```
-- Q1 Find all orders where sales are greater than the average sales. (Subquery)
SELECT *
FROM orders
WHERE Sales > (SELECT AVG(Sales) FROM orders);
```

OUTPUT:

Row_ID	Order_ID	Order_Date	Ship_Date	Ship_Mode	Customer_ID	Product_ID	Sales	Quantity	Discount	Profit
1	CA-2016-152156	11/8/2016	11/11/2016	Second Class	CG-12520	FUR-BO-10001798	261.9600	2	0.0000	41.9136
2	CA-2016-152156	11/8/2016	11/11/2016	Second Class	CG-12520	FUR-CH-10000454	731.9400	3	0.0000	219.5820
4	US-2015-108966	10/11/2015	10/18/2015	Standard Class	SO-20335	FUR-TA-10000577	957.5775	5	0.4500	-383.0310
8	CA-2014-115812	6/9/2014	6/14/2014	Standard Class	BH-11710	TEC-PH-10002275	907.1520	6	0.2000	90.7152
11	CA-2014-115812	6/9/2014	6/14/2014	Standard Class	BH-11710	FUR-TA-10001539	1706.1840	9	0.2000	85.3092
12	CA-2014-115812	6/9/2014	6/14/2014	Standard Class	BH-11710	TEC-PH-10002033	911.4240	4	0.2000	68.3568
14	CA-2016-161389	12/5/2016	12/10/2016	Standard Class	IM-15070	OFF-BI-10003656	407.9760	3	0.2000	132.5922
17	CA-2014-105893	11/11/2014	11/18/2014	Standard Class	PK-19075	OFF-ST-10004186	665.8800	6	0.0000	13.3176
25	CA-2015-106220	6/25/2015	6/30/2015	Standard Class	EB-12970	FUR-TA-10000577	1044.6200	2	0.0000	740.7640

Explanation: This query uses a subquery to calculate the average sales value of all orders in the dataset. The main query then compares each order's sales amount with that average value. Only orders with sales greater than the average are displayed in the result.

Insight: The result highlights above-average transactions. These orders contribute a larger share of revenue and represent high-value sales for the business.

SQL QUERY:

```
-- Q2 Find the highest sales order for each customer. (Subquery)
SELECT *
FROM orders o
WHERE Sales = (SELECT MAX(os.Sales) FROM orders os WHERE os.Customer_ID = o.Customer_ID);
```

OUTPUT:

Row_ID	Order_ID	Order_Date	Ship_Date	Ship_Mode	Customer_ID	Product_ID	Sales	Quantity	Discount	Profit
2	CA-2016-152156	11/8/2016	11/11/2016	Second Class	CG-12520	FUR-CH-10000454	731.9400	3	0.0000	219.5820
4	US-2015-108966	10/11/2015	10/18/2015	Standard Class	SO-20335	FUR-TA-10000577	957.5775	5	0.4500	-383.0310
11	CA-2014-115812	6/9/2014	6/14/2014	Standard Class	BH-11710	FUR-TA-10001539	1706.1840	9	0.2000	85.3092
25	CA-2015-106320	9/25/2015	9/30/2015	Standard Class	EB-13870	FUR-TA-10000577	1044.6300	3	0.0000	240.2649
28	US-2015-150630	9/17/2015	9/21/2015	Standard Class	TB-21520	FUR-BO-10004834	3083.4300	7	0.5000	-1665.0522
36	CA-2016-117590	12/8/2016	12/10/2016	First Class	GH-14485	TEC-PH-10004977	1097.5440	7	0.2000	123.4737
39	CA-2015-117415	12/27/2015	12/31/2015	Standard Class	SN-20710	FUR-BO-10002545	532.3992	3	0.3200	-46.9764
55	CA-2016-105816	12/11/2016	12/17/2016	Standard Class	JM-15265	TEC-PH-10002447	1029.9500	5	0.0000	298.6855
59	CA-2016-111687	6/17/2016	6/18/2016	First Class	TB-21520	FUR-CH-10002068	210.4100	5	0.1000	7.0000

Explanation: This query uses a correlated subquery to find the maximum sales amount for every customer. For each customer, the query compares all of their orders and returns the order with the highest sales value. This ensures that the top purchase of every customer is identified.

Insight: The result shows the largest purchase made by each customer. It helps understand customer spending behavior and identify customers capable of making high-value purchases.

SQL QUERY:

```
-- Q3 Calculate total sales for each customer. (CTE)
WITH CustomerSales AS (SELECT Customer_ID, SUM(Sales) AS Total_Sales
FROM orders
GROUP BY Customer_ID)

SELECT *
FROM CustomerSales;
```

OUTPUT:

	Customer_ID	Total_Sales
▶	CG-12520	1148.7800
	DV-13045	1119.4830
	SO-20335	2602.5755
	BH-11710	6255.3510
	AA-10480	1790.5120
	IM-15070	4930.4740
	HP-14815	1990.3140
	PK-19075	8646.9340
	AG-10270	2582.8980
	70-21025	1402.0440

Explanation: A Common Table Expression (CTE) is used to group orders by customer and calculate the total sales generated by each one. The SUM() function adds the sales values of all orders belonging to a customer. The result provides a consolidated view of customer revenue contribution.

Insight: This analysis helps identify customers who contribute the most revenue. It can be used to recognize valuable customers and support customer-focused business decisions.

SQL QUERY:

```
-- Q4 Find customers whose total sales are above average. (CTE + Subquery)
WITH CustomerSales AS (SELECT Customer_ID, SUM(Sales) AS Total_Sales
FROM orders
GROUP BY Customer_ID)

SELECT *
FROM CustomerSales
WHERE Total_Sales > (SELECT AVG(Total_Sales) FROM CustomerSales);
```

OUTPUT:

	Customer_ID	Total_Sales
▶	BH-11710	6255.3510
	IM-15070	4930.4740
	PK-19075	8646.9340
	TB-21520	4737.4860
	MA-17560	4299.1610
	SN-20710	3127.9592
	LC-16930	4492.9460
	RA-19885	3832.3140
	ES-14080	4657.9240
	VM-16720	4000.4720

Explanation: The CTE first calculates total sales for each customer. A subquery then computes the average of those total sales values across all customers. Finally, only customers whose total sales exceed that average are returned.

Insight: These customers perform better than the overall customer base. They contribute a significant portion of total revenue and can be considered high-value customers.

SQL QUERY:

```
-- Q5 Rank all customers based on total sales. (Window Function)
WITH CustomerSales AS (SELECT Customer_ID, SUM(Sales) AS Total_Sales
FROM orders
GROUP BY Customer_ID)

SELECT Customer_ID, Total_Sales, RANK() OVER (ORDER BY Total_Sales DESC) AS Sales_Rank
FROM CustomerSales;
```

OUTPUT:

	Customer_ID	Total_Sales	Sales_Rank
▶	SM-20320	25043.0500	1
	TC-20980	19052.2180	2
	RB-19360	15117.3390	3
	TA-21385	14595.6200	4
	AB-10105	14473.5710	5
	KL-16645	14175.2290	6
	SC-20095	14142.3340	7
	HL-15040	12873.2980	8
	SE-20110	12209.4380	9
	CC-12370	12170.0770	10

Explanation: This query first calculates total sales for each customer using a CTE. The RANK() window function is then applied to assign ranks based on total sales in descending order. Customers with higher sales receive better ranks.

Insight: Ranking helps compare customer performance in a structured way. It quickly identifies top-performing customers and shows how customers compare against one another.

SQL QUERY:

```
-- Q6 Assign row numbers to each order within a customer. (Window Function + PARTITION BY)
SELECT Customer_ID, Order_ID, Sales,
ROW_NUMBER() OVER (PARTITION BY Customer_ID ORDER BY Sales DESC) AS Row_Num
FROM orders;
```


OUTPUT:

	Customer_ID	Order_ID	Sales	Row_Num
▶	AA-10315	CA-2016-103982	3930.0720	1
	AA-10315	CA-2014-128055	673.5680	2
	AA-10315	CA-2016-103982	431.9760	3
	AA-10315	CA-2017-147039	362.9400	4
	AA-10315	CA-2014-128055	52.9800	5
	AA-10315	CA-2016-103982	41.7200	6
	AA-10315	CA-2015-121391	26.9600	7
	AA-10315	CA-2014-138100	14.9400	8
	AA-10315	CA-2014-138100	14.5600	9
	AA-10315	CA-2017-147039	11.5400	10

Explanation: The ROW_NUMBER() window function is used along with PARTITION BY Customer_ID. This creates a separate sequence of row numbers for each customer. Orders are sorted by sales amount so that the highest-value order receives row number 1.

Insight: The query helps analyze customer purchasing patterns. It can be used to identify the first, second, or third highest-value orders for every customer.

SQL QUERY:

```
-- Q7 Display top 3 customers based on total sales. (Window Function)
WITH CustomerSales AS (SELECT Customer_ID, SUM(Sales) AS Total_Sales
FROM orders
GROUP BY Customer_ID),

RankedCustomers AS (SELECT Customer_ID, Total_Sales,
RANK() OVER (ORDER BY Total_Sales DESC) AS Sales_Rank
FROM CustomerSales)

SELECT *
FROM RankedCustomers
WHERE Sales_Rank <= 3;
```

OUTPUT:

	Customer_ID	Total_Sales	Sales_Rank
▶	SM-20320	25043.0500	1
	TC-20980	19052.2180	2
	RB-19360	15117.3390	3

Explanation: The query first calculates total sales for each customer and then ranks them using the RANK() function. After ranking, only customers whose rank is 3 or less are selected. This returns the top-performing customers based on revenue.

Insight: The top three customers contribute the highest sales among all customers. These customers are important for business growth and customer retention efforts.

● STEP 3: FINAL COMBINED QUERY

SQL QUERY:

```
-- STEP 3
-- Final Combined Query (JOIN + CTE + Window Function)

● WITH CustomerSales AS (SELECT Customer_ID, SUM(Sales) AS Total_Sales
  FROM orders
  GROUP BY Customer_ID)

  SELECT c.Customer_Name, cs.Total_Sales,
  RANK() OVER (ORDER BY cs.Total_Sales DESC) AS Sales_Rank
  FROM CustomerSales cs
  JOIN customers c
  ON cs.Customer_ID = c.Customer_ID;
```

OUTPUT:

	Customer_Name	Total_Sales	Sales_Rank
▶	Sean Miller	25043.0500	1
	Sean Miller	25043.0500	1
	Sean Miller	25043.0500	1
	Sean Miller	25043.0500	1
	Sean Miller	25043.0500	1
	Tamara Chand	19052.2180	6
	Tamara Chand	19052.2180	6
	Tamara Chand	19052.2180	6
	Tamara Chand	19052.2180	6
	Tamara Chand	19052.2180	6
	Raymond Buch	15117.3390	11
	Raymond Buch	15117.3390	11
	Raymond Buch	15117.3390	11

Explanation: This query combines multiple SQL concepts including CTEs, JOINS, and window functions. The CTE calculates total sales for each customer, which is then joined with the customers table to retrieve customer names. The RANK() function assigns ranks based on total sales, creating a complete customer performance report.

Insight: The result provides a summary of customer performance in a single output. It helps compare customers using both their sales contribution and ranking.

- **MINI PROJECT: CUSTOMER SALES INSIGHTS**

SQL QUERY:

```
-- Mini Project: Customer Sales Insights

-- Q1 Who are the top 5 customers?
WITH CustomerSales AS (SELECT Customer_ID, SUM(Sales) AS Total_Sales
FROM orders
GROUP BY Customer_ID)

SELECT *
FROM CustomerSales
ORDER BY Total_Sales DESC
LIMIT 5;
```

OUTPUT:

	Customer_ID	Total_Sales
▶	SM-20320	25043.0500
	TC-20980	19052.2180
	RB-19360	15117.3390
	TA-21385	14595.6200
	AB-10105	14473.5710

Explanation: The query calculates total sales for each customer and sorts them in descending order. The first five records are selected using LIMIT 5. This identifies the customers with the highest sales contribution.

Insight: These customers are the most valuable to the business. They generate the highest revenue and have a major impact on overall sales performance.

SQL QUERY:

```
-- Q2 Who are the bottom 5 customers?  
● WITH CustomerSales AS (SELECT Customer_ID, SUM(Sales) AS Total_Sales  
  FROM orders  
  GROUP BY Customer_ID)  
  
  SELECT *  
  FROM CustomerSales  
  ORDER BY Total_Sales ASC  
  LIMIT 5;
```

OUTPUT:

	Customer_ID	Total_Sales
▶	TS-21085	4.8330
	LD-16855	5.3040
	CJ-11875	16.5200
	MG-18205	16.7390
	RS-19870	22.3280

Explanation: The query calculates total sales for each customer and sorts the results in ascending order. The five customers with the lowest sales values are displayed. This helps identify low-performing customers.

Insight: These customers contribute the least revenue. Their purchasing activity may require further analysis to improve engagement and sales.

SQL QUERY:

```
-- Q3 Which customers made only one order?  
SELECT Customer_ID, COUNT(DISTINCT Order_ID) AS Order_Count  
FROM orders  
GROUP BY Customer_ID  
HAVING COUNT(DISTINCT Order_ID) = 1;
```

OUTPUT:

	Customer_ID	Order_Count
▶	AO-10810	1
	AR-10570	1
	CJ-11875	1
	JC-15385	1
	JR-15700	1
	LD-16855	1
	MG-18205	1
	PH-18790	1
	RE-19405	1
	RM-19750	1
	SM-20905	1
	TC-21145	1

Explanation: The query counts the number of distinct orders placed by each customer. Customers with exactly one order are filtered using the HAVING clause. This identifies one-time buyers in the dataset.

Insight: The result shows customers who purchased only once. These customers may have lower engagement compared to repeat customers.

SQL QUERY:

```
-- Q4 Which customers have above-average sales?  
WITH CustomerSales AS (SELECT Customer_ID, SUM(Sales) AS Total_Sales  
FROM orders  
GROUP BY Customer_ID)  
  
SELECT *  
FROM CustomerSales  
WHERE Total_Sales > (SELECT AVG(Total_Sales) FROM CustomerSales);
```

OUTPUT:

	Customer_ID	Total_Sales
▶	BH-11710	6255.3510
	IM-15070	4930.4740
	PK-19075	8646.9340
	TB-21520	4737.4860
	MA-17560	4299.1610
	SN-20710	3127.9592
	LC-16930	4492.9460
	RA-19885	3832.3140
	ES-14080	4657.9240
	KM-16720	4909.4720
	KD-16270	8282.3580
	TE-15745	6760.8150

Explanation: The query first calculates total sales for each customer and then finds the average customer sales value. Customers whose total sales exceed this average are selected. This helps separate higher-performing customers from the rest.

Insight: These customers contribute more revenue than the average customer. They form an important segment for business growth and customer loyalty programs.

SQL QUERY:

```
-- Q5 What is the highest order value per customer?  
SELECT Customer_ID, MAX(Sales) AS Highest_Order_Value  
FROM orders  
GROUP BY Customer_ID;
```

OUTPUT:

	Customer_ID	Highest_Order_Value
►	CG-12520	731.9400
	DV-13045	721.8750
	SO-20335	957.5775
	BH-11710	1706.1840
	AA-10480	479.9700
	IM-15070	1819.8600
	HP-14815	1242.9000
	PK-19075	4305.5520
	AG-10270	1336.4400
	ZD-21925	823.9600
	KB-16585	1013.8320
	SF-20065	254.9000

Explanation: The query groups data by customer and uses the MAX() function to find the highest sales value recorded for each customer. It returns one maximum order amount for every customer. This highlights each customer's largest transaction.

Insight: The result shows the peak spending amount for every customer. It helps understand customer purchasing capacity and spending potential.

● CONCLUSION

This assignment demonstrated the use of advanced SQL concepts such as Subqueries, Common Table Expressions (CTEs), and Window Functions on the Superstore dataset. The dataset was organized into separate customers, products, and orders tables to improve data management and analysis. Various business questions were answered, including identifying high-value orders, top customers, above-average customers, and customer purchase patterns. Window functions such as RANK() and ROW_NUMBER() were used to perform customer ranking and order-level analysis efficiently. The final combined query integrated multiple SQL concepts to generate a comprehensive customer performance report. Overall, this assignment provided practical experience in using SQL for sales analysis and customer insights, helping transform raw transactional data into meaningful business information.